

Abstract geometric lines in the top-left corner of the slide, consisting of several overlapping, irregular polygons and lines in a light gray color.

SWE4913: COMMUNICATION IN DISTRIBUTED ARCHITECTURE

Optimizing Point-to-Point Data Transfer
Eddie Oendo

AGENDA

The Stage2 Context

Existing Technology

Understanding the Infrastructure

Building Broadcasters

Technical Hurdles



THE STAGE2 CONTEXT



OPERATIONS

What is an operation

An executable unit of work used to perform a specific task:

- **Containerized:** Run in isolated Docker environments.
- **JSON-Driven:** Inputs, outputs, and parameters are defined in a blueprint.
- **Connected:** linked with one another, passing outputs as inputs for later tasks

The Orchestration Challenge:

How do we move gigabytes of data between these isolated "islands" without the CPU overhead of traditional file copying?

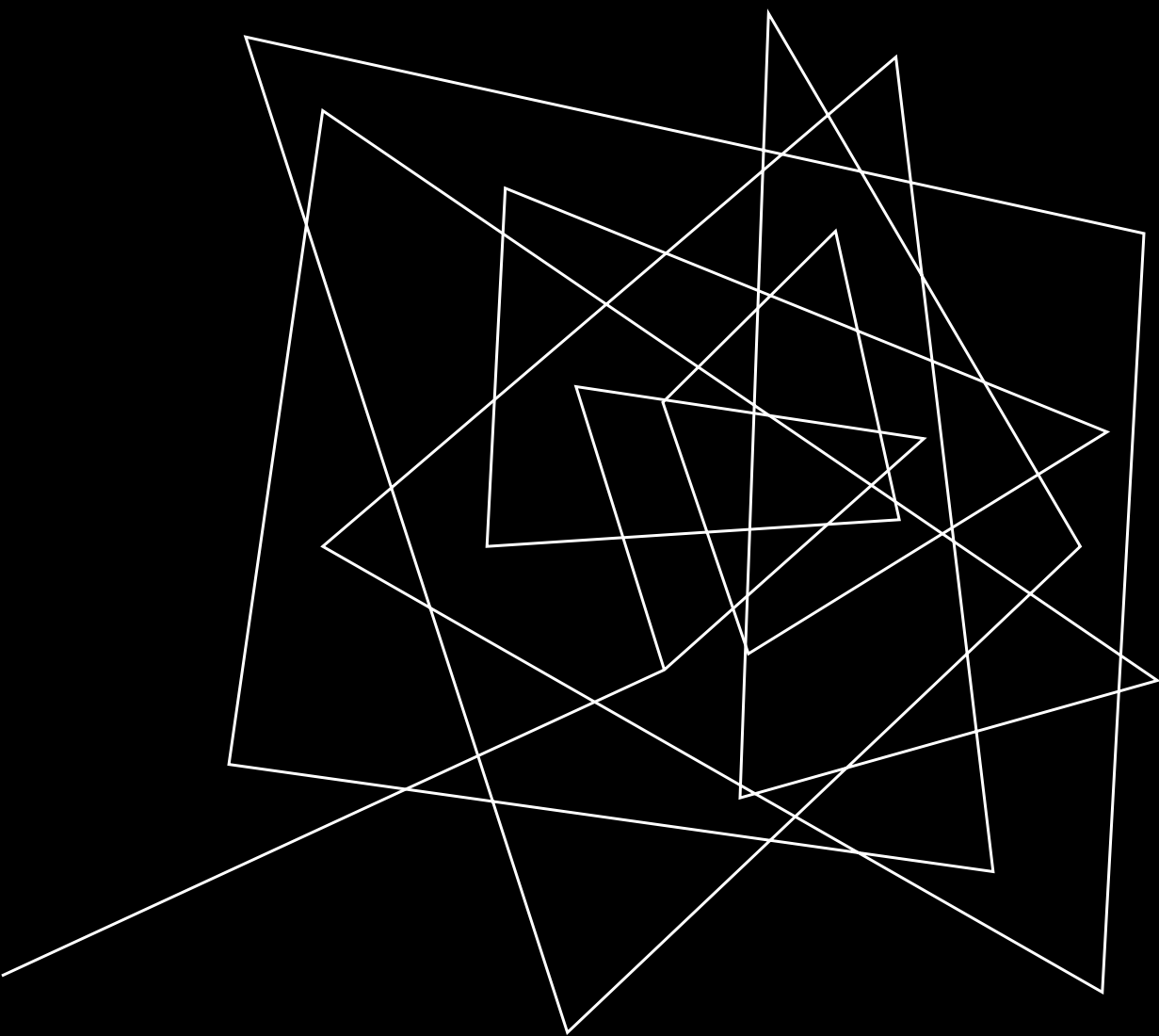
THE BOTTLENECK: "ROUTING OUTPUTS"

Stage2 Workflow Logic: One operation's output is often the input for *multiple* downstream nodes (Multiplicity).

The Problem: Traditional sequential copying creates a linear delay.

- **Copy 1 to Node A... then Copy 2 to Node B...**

The Solution: An integrated Broadcaster that duplicates data, allowing all downstream nodes to receive data simultaneously.



EXISTING
TECHNOLOGY

MESSAGE PASSING INTERFACE (MPI)

The industry standard for communication in distributed-memory systems

- **Abstraction of network complexity:** through use of communicators and groups to isolate process communication.
- **Bypassing hardware heterogeneity:** Uses opaque data types and handles translation between different machines
- **Provides a layered Architecture:** (MCA) allows for runtime selection of transport layers. (PML) handles message matching and ordering, while (BTL) interfaces with the physical hardware (e.g., InfiniBand via UCX or Ethernet via TCP)





THE "MEMORY WALL"

MPI introduces a significant "copy tax" where the CPU's ability to process data outstrips the memory bus's ability to move it.

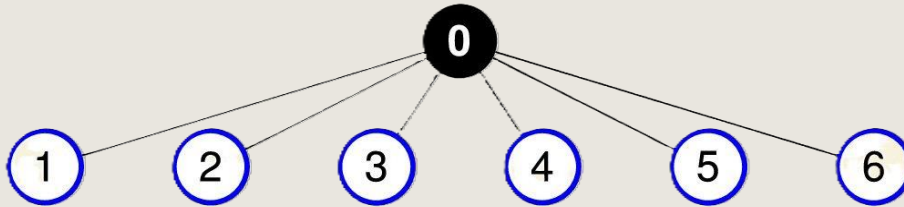
- **Application Buffer to MPI Buffer:** Data is copied into an internal buffer.
- **User-space to Kernel-space:** The data is passed to the OS network stack.
- **Kernel to Hardware:** The NIC finally pulls the data from the kernel buffer.



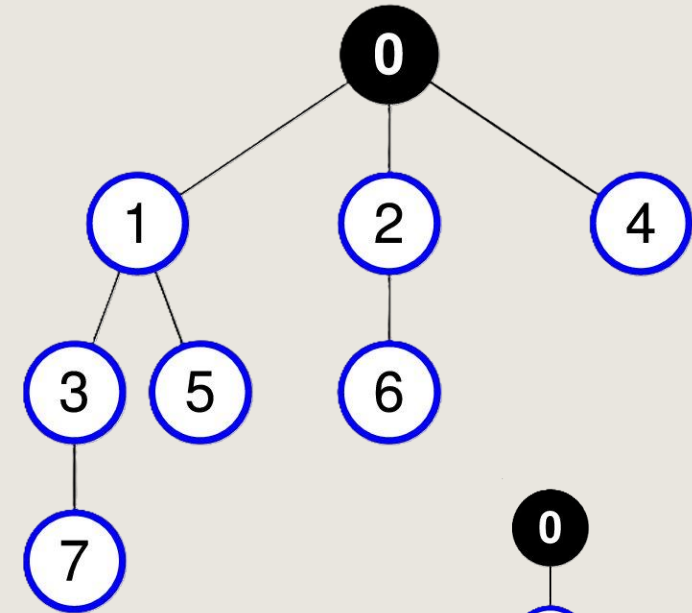
UNDERSTANDING THE INFRASTRUCTURE

MPI BROADCASTING ALGORITHMS

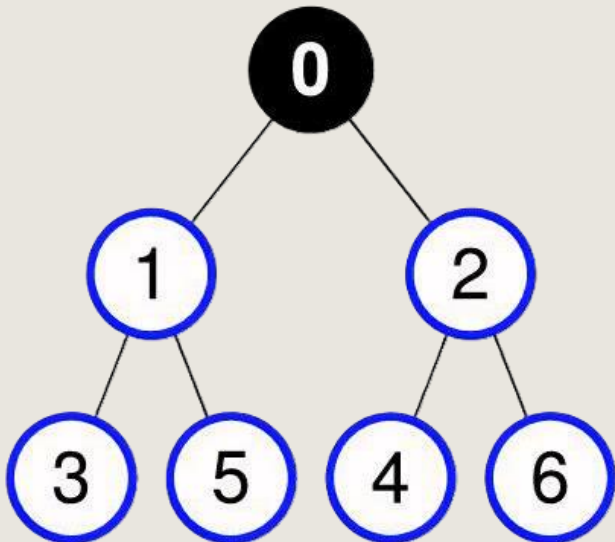
Flat tree algorithm:



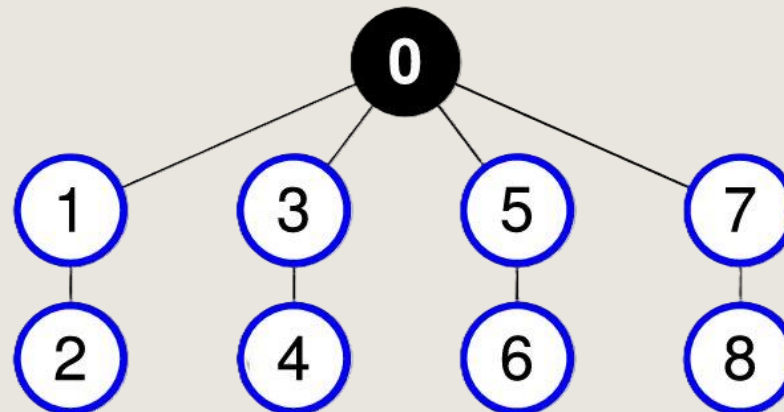
Balanced-binomial tree:



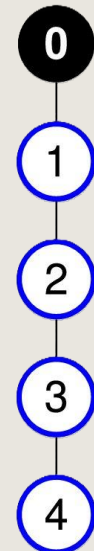
Binary tree:



K-Chain:



Chain tree:





HTTP ERROR HANDLING

Resumable / Partial Downloads

Allows retrieval of specific portions of a resource rather than downloading the entire file using headers to:

- Specify data ranges
- Describes where the segment fits within the full resource
- Indicate if the data has been modified since last download

Resumable / Multipart Uploads

Reliably transfer large files over unstable or bandwidth-limited networks:

- Sender send a POST request and receives an upload-id
- File is sliced into chunks uploaded independently along with its part-number and the upload-id.
- On completion sender sends a list of Etags for verification and reconstruction

NETWORK SPEED LIMITERS

Latency

Data transmission time from the origin to the destination. This delay encompasses several factors:

- Processing delay
- Queueing delay
- Transmission delay
- Propagation delay

Jitter

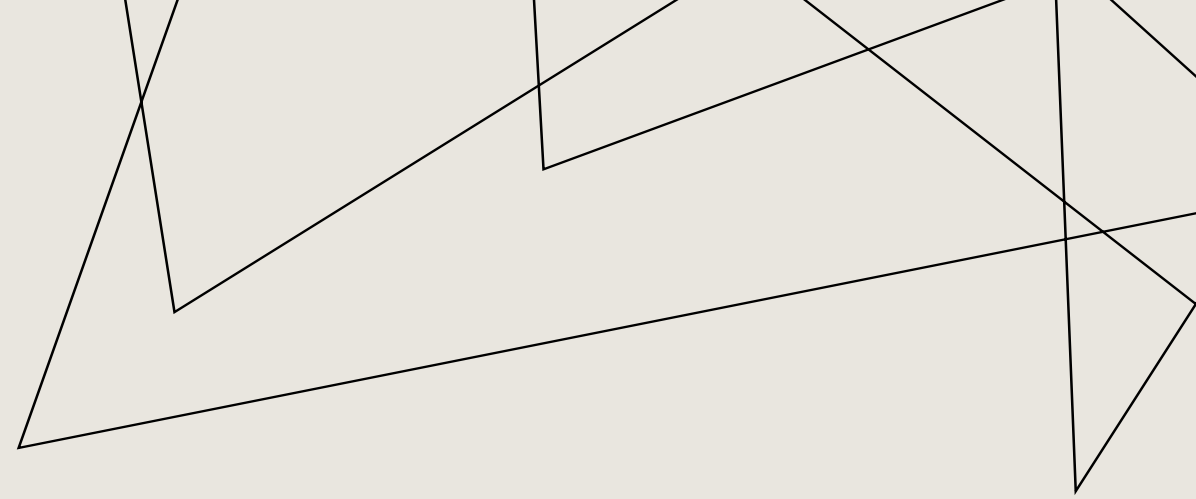
The measure of the variation of latency over time, measured in milliseconds. Affected by:

- Network congestion
- Routing changes
- Variable path lengths
- Equipment limitation

LOGIC OF CHUNKING

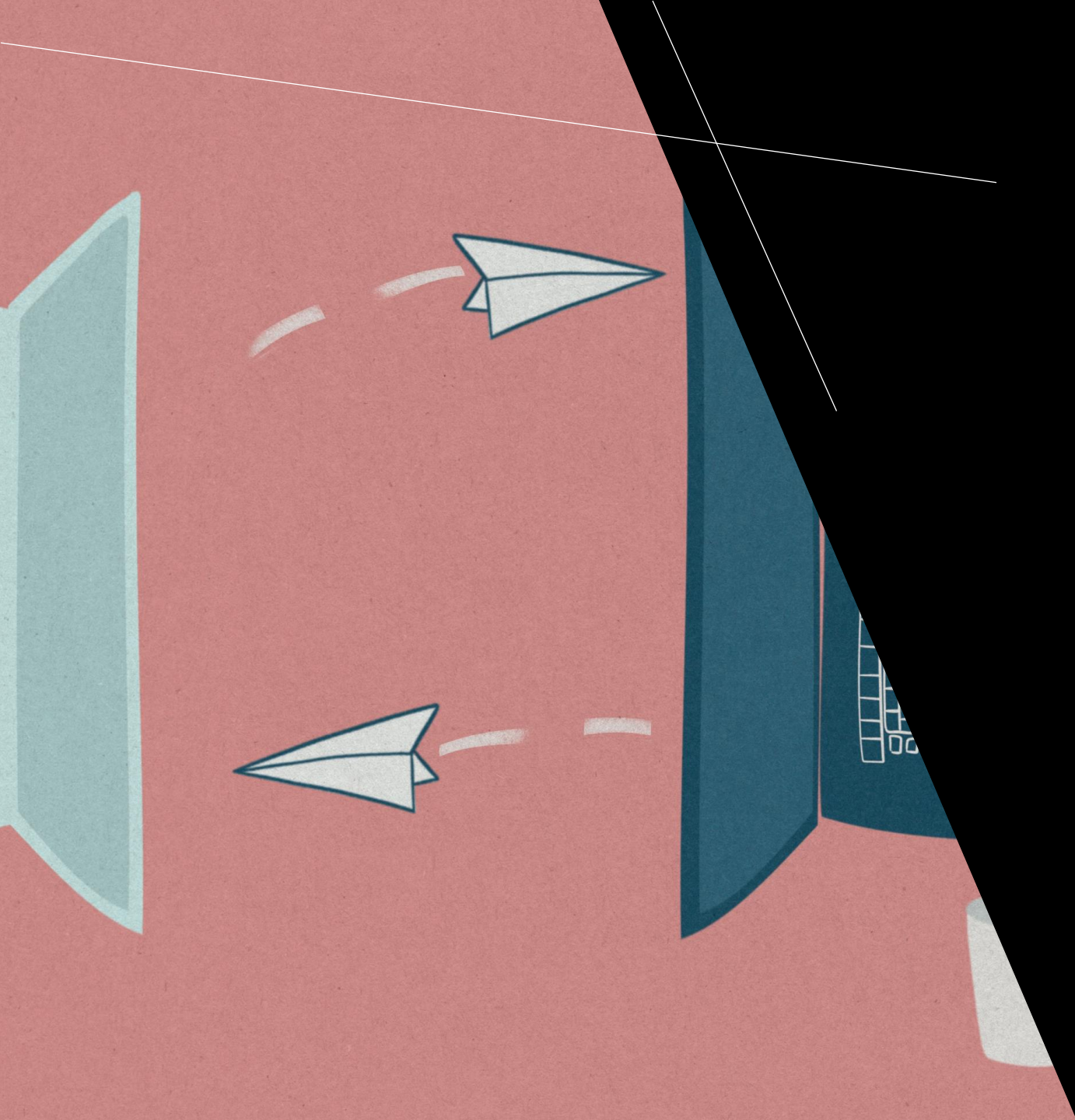
- To saturate a network link between two nodes, data cannot be sent as a single monolithic block.
- Splitting large data sets or messages into smaller, manageable pieces, enhances performance and efficiency in data transmission.
- Each chunk is accompanied by metadata that provides details about the chunk, such as its size and content type.
- As the chunks are received, they are immediately processed, allowing for a more efficient transfer of data.
- Useful in scenarios where the total size of the response is unknown or cannot be determined in advance

CONCURRENCY ARCHITECTURE



Use of asynchronous programming to avoid the OS' context switching overhead by processing much of the I/O in a single thread.

- **Iterative:** This processes one request after another.
- **Forking:** This creates a new process for each request that needs to be served.
- **Pre-forked:** Creates a pool of processes that are assigned requests as they come in, avoids the overhead of having to create them at runtime.
- **Threaded:** This spawns a new thread every time a request needs to be processed.
- **Pre-threaded:** Similar to pre-forked but with threads.
- **Poll:** A single threaded and uses the poll(2) system call to multiplex between requests.



BUILDING BROADCASTERS

PREADV / PWRITEV

- A thread safe version of readv() system call.
- Used as the baseline for the broadcasters
- Fills up iovec buffers from the source file and writes to receiver file repeatedly till file is fully read.
- Utilizes OpenMP to implement the MPI Binary-tree broadcasting algorithm for parallel transmissions.

PERFORMANCE

- Highest variance / jitter, with throughput starting from 137 MB/s before settling near 450 MB/s.
- This fluctuations are due to the overhead of context switching between user/kernel space and synchronization
- Heavily affected by the "copy tax" during the memory allocation and data transfer phases.

IO_URING

- Linux-specific API for asynchronous I/O.
- It allows submission of one or more I/O requests without blocking the calling process.
- Removes the limitations of the current system calls used to monitor file descriptors and let you know when one or more of them are ready.

IO_URING(READV)

- Similar to readv in terms of copying data into iovecs.
- Significantly reduces context switching as it uses a shared-memory ring buffer architecture for user/kernel space communication .
- Allows the application to place multiple I/O requests in a Submission Queue (SQ) and move on to other tasks

PERFORMANCE

- A significant improvement in stability and a 31% increase in average throughput with throughput averaging 433 MB/s.
- Validating the efficiency of `io_uring`'s submission queue mechanism for handling multiple I/O vectors without the overhead of threading
- Still, it is heavily affected by the "copy tax" during the memory allocation and data transfer phases.

IO_URING(SPLICE)

- Utilizes the splice() system call which moves data between two file descriptors without copying between kernel address space and user address space.
- Kernel creates 'copies' of reference-counted pointer to pages in the memory; passes it the file and increase the count.
- Avoiding duplicating data to the new location, thus alleviating the "copy tax".
- Uses pipes to move data between files and each call consumes the data in the pipe.

PERFORMANCE

- Outperformed all buffer-based methods averaging a throughput of 461.91 MB/s with peaks of 600 MB/s
- Validating by reducing the need for data duplication, the system avoids the CPU cycles needed to copy data onto user-space memory.
- Due to the consumption of data with each call, it enforces a need to retransmit the same data to facilitate broadcasting inhibiting true parallelization.

IO_URING(TEE/SPLICE)

- Incorporates the tee() system call which works similarly to splice() but does not consume the data it duplicates.
- Therefore, that data can be copied by a subsequent splice() call.
- This enables a single "read" from the source file to a pipe to be used to feed multiple "writes" to destination files simultaneously.

PERFORMANCE

- Emerged as the most efficient strategy, achieving an average throughput of 620.62 MB/s. A 31% improvement over splice() and an 81% improvement over the baseline.
- Outperformed all earlier methods with peaks of 1000+ MB/s

COMPARATIVE ANALYSIS

Summarizes OF the average throughput and completion times across all trials.*

STRATEGY	AVG THROUGHPUT (MB/S)	AVG TIME (s)	EFFICIENCY RANK
OMP preadv/pwritev	432.12	0.0624	4
IO_uring readv/writev	439.29	0.0387	3
IO_uring Splice	461.91	0.0329	2
IO_uring Tee / Splice	611.25	0.0273	1

***Disclaimer:** research was conducted on a Windows Subsystem for Linux (WSL) environment thus, performance is highly affected by virtualization and higher averages can be achieved on dedicated hardware.

LEVERAGING IO_URING FOR RELIABILITY

- Every `io_uring` request carries a `user_data` field that is returned untouched in the Completion Queue Event (CQE).
- Instead of passing simple integers, we pass pointers to metadata structures or bit-packed descriptors containing:
 - Target File Descriptor (FD).
 - Exact Byte Offset of the chunk.
 - Stage 2 Operation ID.
- If a network or disk failure occurs, the Stage2 Orchestrator can restart the broadcaster, passing the last successful offset as a parameter to the next `stage2_interface.py` call.



TECHNICAL HURDLES

PIPE CAPACITY AND DEADLOCKS

- Pipe Deadlock is when multiple process contest for access of the same pipe at the same time causing the program to stall.
- Due to the speed of execution of the asynchronous reads, the program would complete a broadcast operation before new data was written to the pipe leading to a deadlock.
- `fcntl()` was utilized with `F_SETPIPE_SZ` to expand pipe buffers to 1 MB, maximizes the "flight time" of data, ensuring the hardware remains saturated between `io_uring` submissions.

LINKED-CHAIN FRAGILITY

- Due to its asynchronous nature, `io_uring` requests have a tendency for out-of-order execution.
- Kernel might attempt to consume data via a destination splice before the tee duplication is complete, leading to data corruption or `EAGAIN` errors.
- A singular asynchronous batch submission was attempted using `IOSQE_IO_LINK`. However, this resulted in widespread `ECANCELED` errors.
- If any single destination in the chain experienced a short-write or stall, the kernel would discard the entire subsequent broadcast for all other nodes.

LINKED-CHAIN FRAGILITY

We addressed this by implementing a **Hybrid-Link Submission Model**.

1. The initial source → master-pipe splice is submitted as an independent, unlinked SQE.
2. For the first $N - 1$ destinations, we submit linked pairs of (tee → splice). This ensures the destination write only triggers upon a successful duplication and remains asynchronous.
3. The transmission to the N th destination is submitted as a standalone splice to consume the data in the master-pipe.



QUESTIONS

SUMMARY OF THE STUDY

- **Pipelined Orchestration:** The Broadcaster enables downstream operations to begin processing data before source operations have fully finished, creating a truly continuous workflow.
- **Resource Optimization:** By eliminating the "User-Space Data Tax," we free up CPU and Memory resources for the actual scientific computations running inside the containers.

This research provides a scalable, fault-tolerant foundation for the next generation of Stage2 workflow orchestration..

A series of white, thin, overlapping geometric lines on a black background, creating a complex, abstract pattern on the left side of the slide.

THANK YOU

Special thanks to Justen Di Ruscio